



BASES DE DATOS AVANZADAS

JOSE SAUL DE LIRA MIRAMONTES

JOSÉ FERNANDO MARTÍNEZ PACHECO

MATRICULA: 385795

26/ AGOSTO/ 2026

APLICACIÓN DE BASE DE DATOS CON PYTHON

Actividad 3: Aplicación de Base de Datos con Python

1. Objetivo

Desarrollar una aplicación en Python que permita realizar operaciones CRUD (Create, Read, Update y Delete) sobre la tabla JOBS de una base de datos Oracle, utilizando FreeSQL como herramienta para la gestión de la base de datos.

2. Herramientas utilizadas

Para el desarrollo de la actividad se utilizaron las siguientes herramientas:

- Python 3.12.1
- Oracle Database
- FreeSQL
- Biblioteca oracledb
- Biblioteca python-dotenv
- GitHub Codespaces

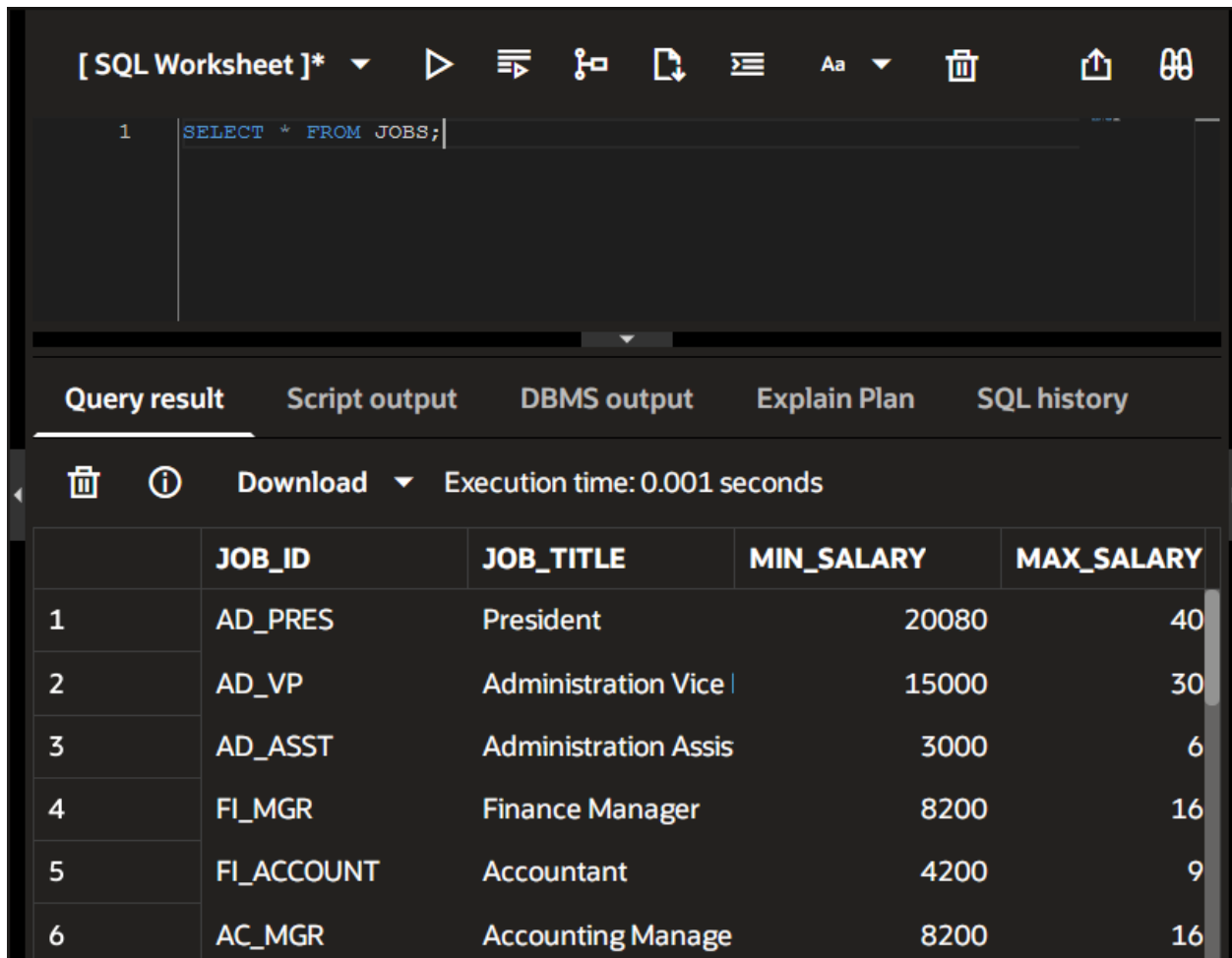
3. Creación de la tabla JOBS

Para realizar la actividad se creó la tabla JOBS en el esquema propio de FreeSQL, utilizando la estructura indicada en las instrucciones:

```
CREATE TABLE jobs(  
    job_id VARCHAR2(10),  
    job_title VARCHAR2(35) CONSTRAINT job_title_nn NOT NULL,  
    min_salary NUMBER(6),  
    max_salary NUMBER(6),  
    CONSTRAINT job_id_pk PRIMARY KEY(job_id)  
);
```

La tabla está formada por los siguientes campos:

- Campo Tipo
- JOB_ID VARCHAR2(10)
- JOB_TITLE VARCHAR2(35)
- MIN_SALARY NUMBER(6)
- MAX_SALARY NUMBER(6)



The screenshot shows an SQL Worksheet interface. At the top, there is a toolbar with various icons for execution, formatting, and window management. Below the toolbar, a text area contains the SQL query: `SELECT * FROM JOBS;`. Below the query area, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is active, displaying a table with 5 columns: 'JOB_ID', 'JOB_TITLE', 'MIN_SALARY', and 'MAX_SALARY'. The table contains 6 rows of data. Above the table, there is a 'Download' button and a status message 'Execution time: 0.001 seconds'.

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
1	AD_PRES	President	20080	40
2	AD_VP	Administration Vice	15000	30
3	AD_ASST	Administration Assis	3000	6
4	FI_MGR	Finance Manager	8200	16
5	FI_ACCOUNT	Accountant	4200	9
6	AC_MGR	Accounting Manage	8200	16

3.1 Consideración sobre la tabla HR.JOBS

Durante la implementación se realizó inicialmente una prueba de conexión y consulta sobre la tabla original HR.JOBS proporcionada por Oracle. La consulta de los registros se realizó correctamente; sin embargo, al intentar realizar una operación INSERT,

Oracle mostró el error ORA-41900, indicando que el usuario utilizado no contaba con privilegios de inserción sobre dicha tabla.

Se verificaron los privilegios disponibles y se comprobó que el acceso otorgado a HR.JOBS era únicamente de lectura (READ). Por este motivo, no fue posible realizar directamente las operaciones INSERT, UPDATE y DELETE sobre la tabla original.

Para poder implementar las cuatro operaciones CRUD y mantener la estructura indicada en la actividad, se creó en el esquema propio la tabla JOBS utilizando exactamente la estructura proporcionada por el profesor.

Posteriormente, se cargaron en esta tabla los 19 registros de referencia de HR.JOBS. De esta manera, la aplicación pudo trabajar con una tabla propia que mantiene la misma estructura y datos de referencia, pero que permite realizar las operaciones de modificación necesarias para el CRUD.

4. Conexión de Python con Oracle

Para establecer la conexión entre Python y Oracle se utilizó la biblioteca oracledb.

Los datos de acceso se almacenaron en un archivo .env, evitando colocar directamente la contraseña dentro del código fuente.

La conexión se encuentra en el archivo database.py, mediante la función conectar():

```
def conectar():
```

```
    dsn = (  
        "(description="   
        "(address=(protocol=tcps)(port=2484)(host=db.freeseql.com))"   
        "(connect_data=(service_name=26ai_un3c1)))"   
    )
```

```
connection = oracledb.connect(  
    user=os.getenv("DB_USER"),  
    password=os.getenv("DB_PASSWORD"),  
    dsn=dsn  
)  
  
return connection
```

La función permite establecer la conexión con Oracle y posteriormente ser utilizada por las operaciones del programa.

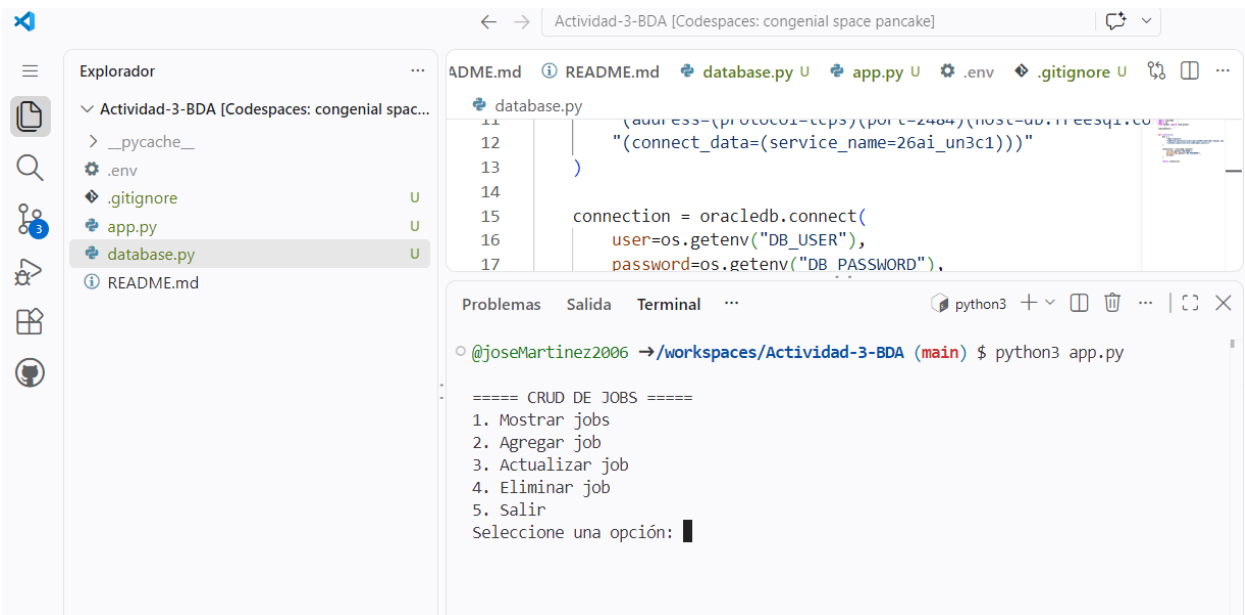
5. Desarrollo de la aplicación

La aplicación fue desarrollada mediante un menú que permite seleccionar la operación que se desea realizar.

===== CRUD DE JOBS =====

1. Mostrar jobs
2. Agregar job
3. Actualizar job
4. Eliminar job
5. Salir

Seleccione una opción:



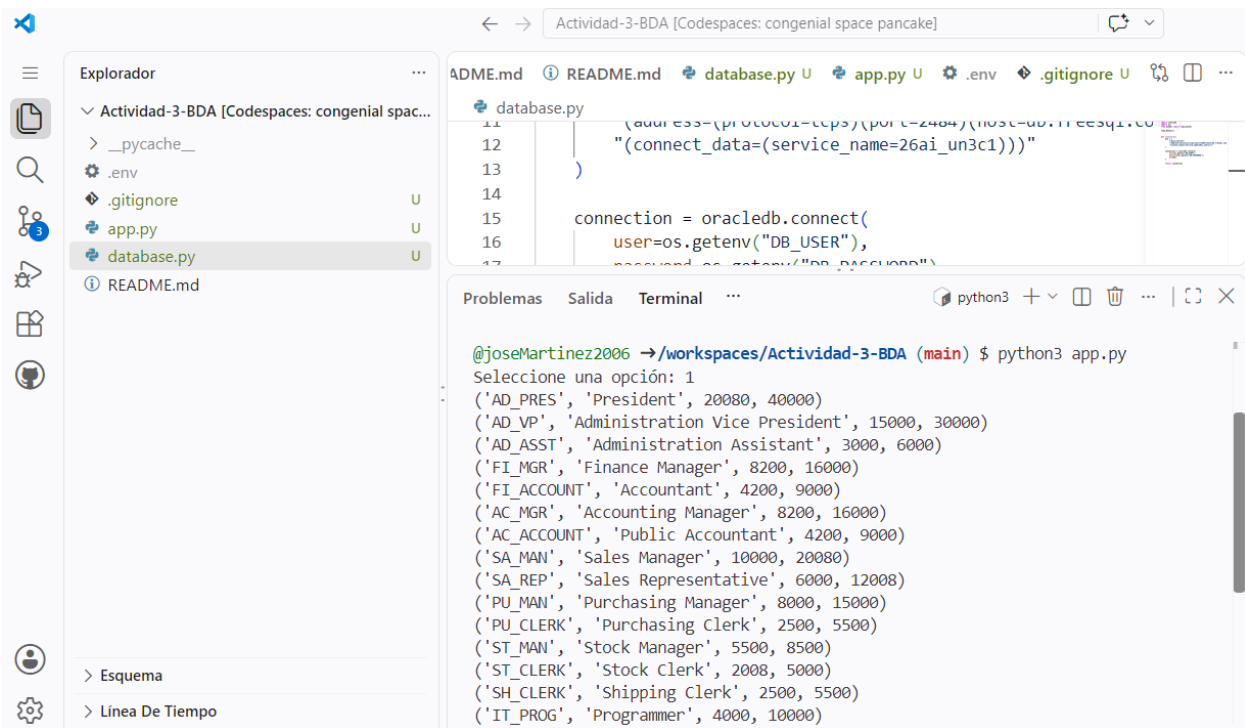
- **5.1 Read — Consultar registros**

La opción 1 permite consultar los registros almacenados en la tabla JOBS.

Para realizar esta operación se utiliza la sentencia:

```
SELECT * FROM JOBS;
```

Los resultados obtenidos de la base de datos son almacenados y posteriormente mostrados en la consola mediante un ciclo.



- **5.2 Create — Agregar un registro**

La opción 2 permite agregar un nuevo registro a la tabla.

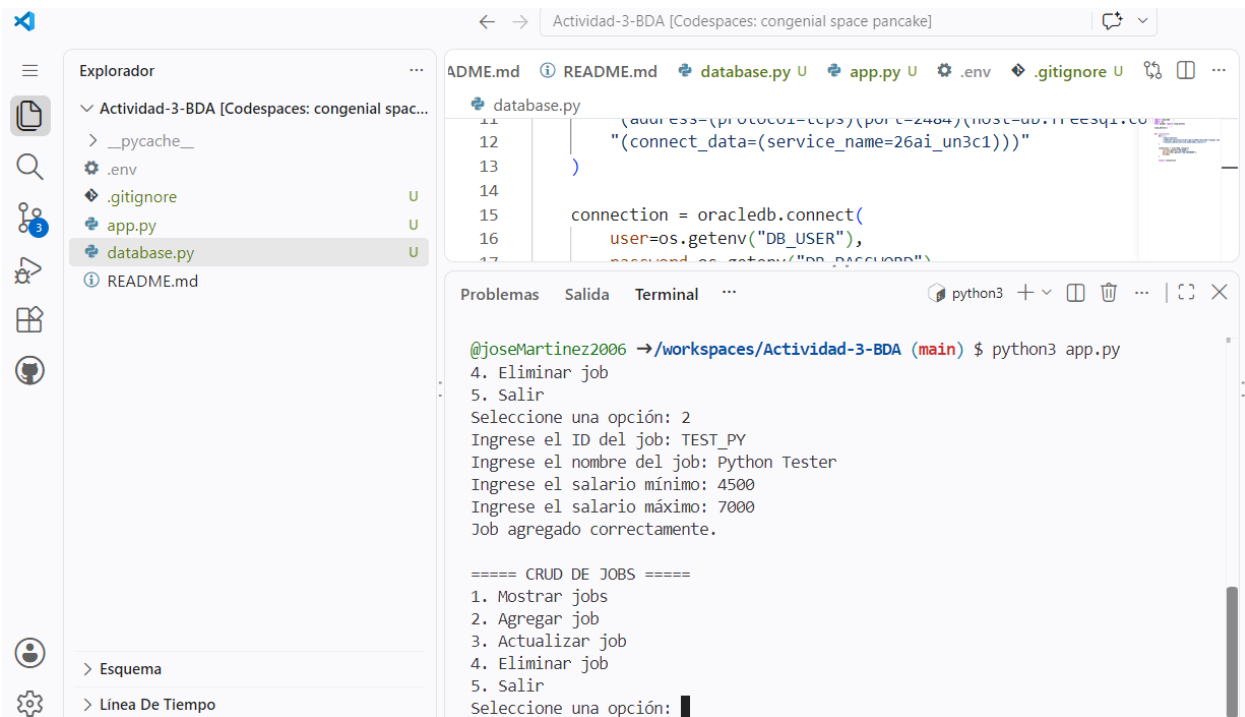
Para realizar esta operación se utiliza:

```
INSERT INTO JOBS (JOB_ID, JOB_TITLE, MIN_SALARY, MAX_SALARY)
```

```
VALUES (:1, :2, :3, :4);
```

El programa solicita al usuario el ID, nombre del job, salario mínimo y salario máximo.

Durante las pruebas se utilizó un registro temporal para comprobar que la operación funcionara correctamente.



• 5.3 Update — Actualizar un registro

La opción 3 permite modificar la información de un registro existente.

La operación se realiza mediante:

UPDATE JOBS

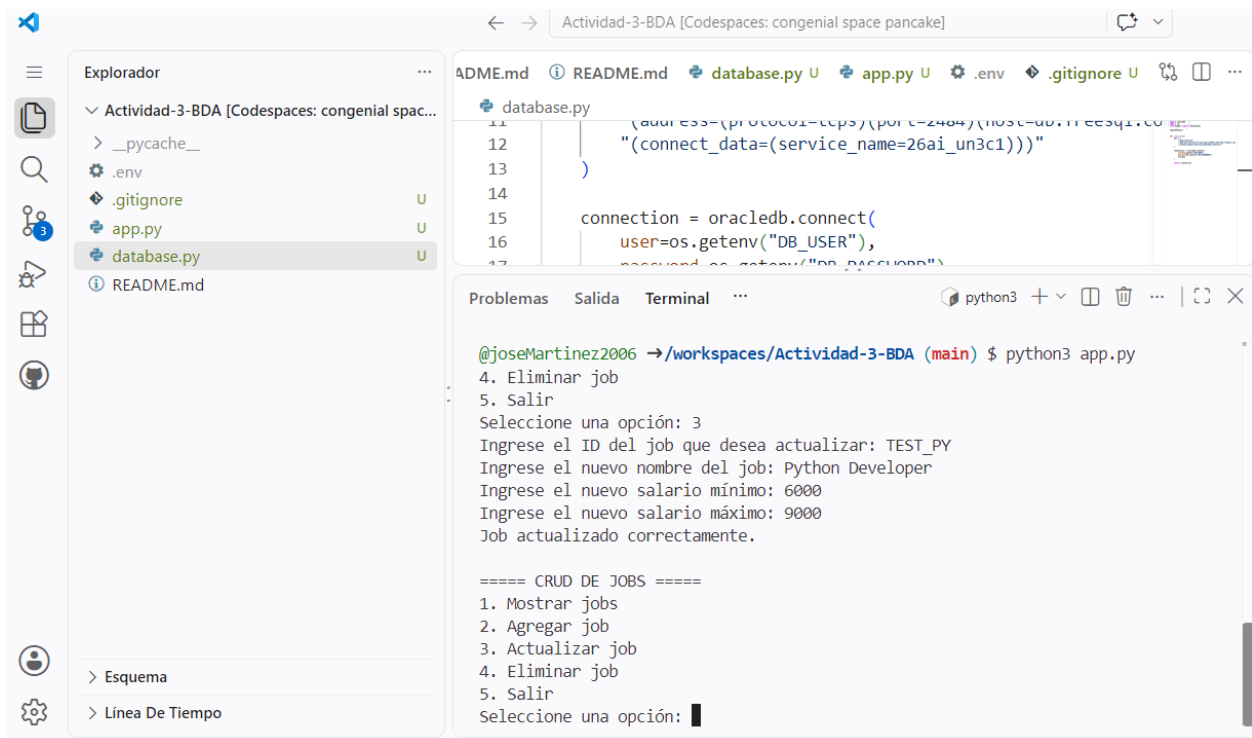
SET JOB_TITLE = :1,

MIN_SALARY = :2,

MAX_SALARY = :3

WHERE JOB_ID = :4;

Para comprobar su funcionamiento se modificó el registro utilizado durante la prueba de Create.



- **5.4 Delete — Eliminar un registro**

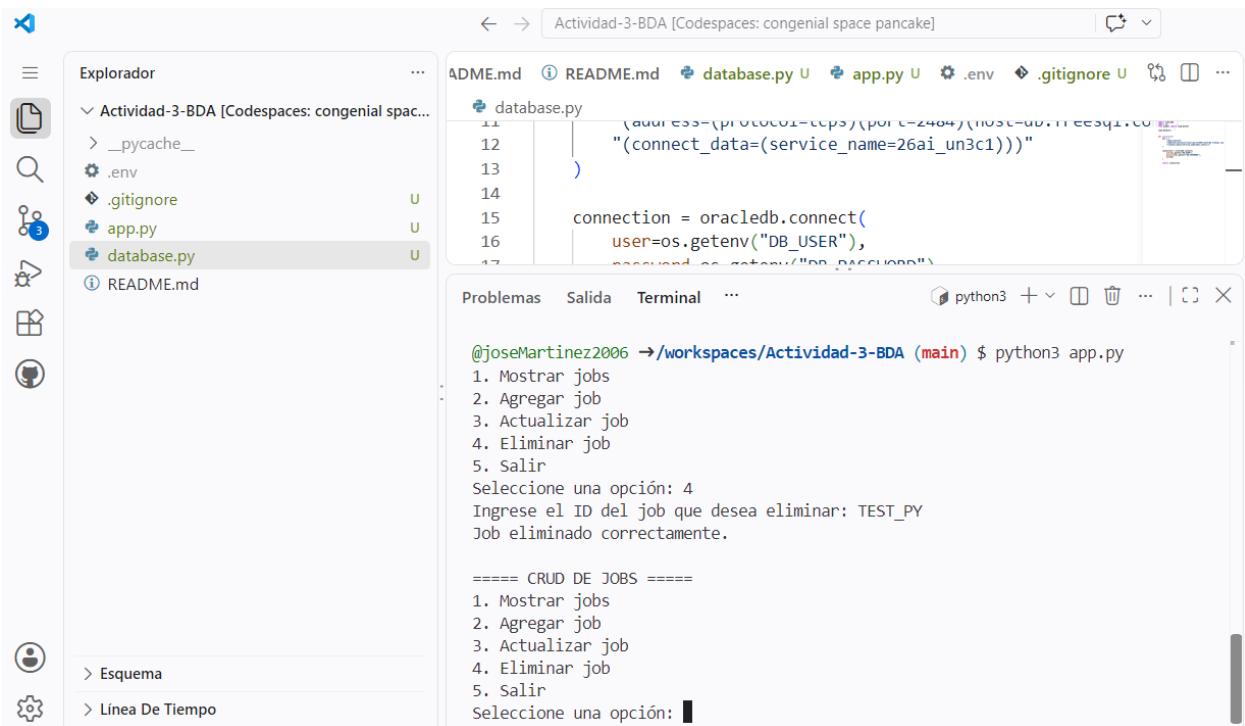
La opción 4 permite eliminar un registro de la tabla utilizando su JOB_ID.

La sentencia utilizada es:

DELETE FROM JOBS

WHERE JOB_ID = :1;

El registro utilizado para las pruebas fue eliminado después de comprobar el funcionamiento de la operación.



6. Estructura del proyecto

El proyecto está organizado de la siguiente manera:

Actividad-3-BDA/

- ├── app.py
- ├── database.py
- ├── .gitignore
- └── README.md

- **app.py**

Contiene la aplicación principal y las funciones correspondientes a las operaciones CRUD:

Mostrar registros.

Agregar registros.

Actualizar registros.

Eliminar registros.

```
from database import conectar
```

```
def mostrar_jobs():
    conexion = conectar()
    cursor = conexion.cursor()

    cursor.execute("SELECT * FROM JOBS")

    jobs = cursor.fetchall()

    if len(jobs) == 0:
        print("No hay jobs registrados.")
    else:
        for job in jobs:
            print(job)

    cursor.close()
    conexion.close()

def agregar_job():
    job_id = input("Ingrese el ID del job: ")
    job_title = input("Ingrese el nombre del job: ")
    min_salary = int(input("Ingrese el salario mínimo: "))
    max_salary = int(input("Ingrese el salario máximo: "))

    conexion = conectar()
    cursor = conexion.cursor()

    sql = """
        INSERT INTO JOBS (JOB_ID, JOB_TITLE, MIN_SALARY, MAX_SALARY)
        VALUES (:1, :2, :3, :4)
    """

    cursor.execute(sql, (job_id, job_title, min_salary, max_salary))

    conexion.commit()

    print("Job agregado correctamente.")
```

```
cursor.close()
conexion.close()
```

```
def actualizar_job():
    job_id = input("Ingrese el ID del job que desea actualizar: ")
    job_title = input("Ingrese el nuevo nombre del job: ")
    min_salary = int(input("Ingrese el nuevo salario mínimo: "))
    max_salary = int(input("Ingrese el nuevo salario máximo: "))

    conexion = conectar()
    cursor = conexion.cursor()

    sql = """
        UPDATE JOBS
        SET JOB_TITLE = :1,
            MIN_SALARY = :2,
            MAX_SALARY = :3
        WHERE JOB_ID = :4
    """

    cursor.execute(sql, (job_title, min_salary, max_salary, job_id))

    conexion.commit()

    print("Job actualizado correctamente.")

    cursor.close()
    conexion.close()
```

```
def eliminar_job():
    job_id = input("Ingrese el ID del job que desea eliminar: ")

    conexion = conectar()
    cursor = conexion.cursor()

    sql = """
        DELETE FROM JOBS
        WHERE JOB_ID = :1
    """

    cursor.execute(sql, (job_id,))

    conexion.commit()
```

```

print("Job eliminado correctamente.")

cursor.close()
conexion.close()

while True:
    print("\n===== CRUD DE JOBS =====")
    print("1. Mostrar jobs")
    print("2. Agregar job")
    print("3. Actualizar job")
    print("4. Eliminar job")
    print("5. Salir")

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        mostrar_jobs()

    elif opcion == "2":
        agregar_job()

    elif opcion == "3":
        actualizar_job()

    elif opcion == "4":
        eliminar_job()

    elif opcion == "5":
        print("Programa finalizado.")
        break

    else:
        print("Opción no válida.")

```

- **database.py**

Contiene la función encargada de establecer la conexión entre Python y Oracle.

```

import oracledb
import os
from dotenv import load_dotenv

```

```
load_dotenv()
```

```
def conectar():  
    dsn = (  
        "(description="   
        "(address=(protocol=tcps)(port=2484)(host=db.freesql.com))"   
        "(connect_data=(service_name=26ai_un3c1))"   
    )  
  
    connection = oracledb.connect(  
        user=os.getenv("DB_USER"),  
        password=os.getenv("DB_PASSWORD"),  
        dsn=dsn  
    )  
  
    return connection
```

- **.gitignore**

Se utiliza para evitar que archivos con información privada o archivos temporales sean incluidos en el repositorio.

En particular:

.env

__pycache__/

El archivo .env contiene las credenciales de conexión y no se incluye en el repositorio.

7. Ejecución de la aplicación

Para ejecutar la aplicación desde GitHub Codespaces se utiliza el siguiente comando:

```
python3 app.py
```

Al ejecutar el programa se muestra el menú principal, desde el cual se pueden seleccionar las diferentes operaciones CRUD.

Durante las pruebas se comprobó el funcionamiento de las cuatro operaciones:

- **Create:** se agregó correctamente un nuevo registro.
- **Read:** se mostraron correctamente los registros almacenados.
- **Update:** se modificó correctamente un registro existente.
- **Delete:** se eliminó correctamente el registro utilizado para la prueba.

Las evidencias de estas operaciones se muestran en las figuras correspondientes de este reporte.

8. Conclusión

La actividad permitió desarrollar una aplicación en Python capaz de conectarse a una base de datos Oracle mediante FreeSQL y realizar las operaciones básicas de un sistema CRUD.

Durante el desarrollo se comprobó la conexión entre Python y Oracle y se implementaron correctamente las operaciones de crear, consultar, actualizar y eliminar registros sobre la tabla JOBS.

También se identificó que la tabla original HR.JOBS disponible en FreeSQL contaba únicamente con permisos de lectura para el usuario utilizado. Por este motivo, se creó una tabla JOBS propia utilizando la estructura indicada en la actividad y se cargaron los registros de referencia de HR.JOBS, permitiendo realizar correctamente las pruebas de las operaciones CRUD.

Finalmente, se utilizó un archivo .env para mantener las credenciales de conexión separadas del código fuente, evitando exponer información sensible en el repositorio.